



Senior MLOps / LLMOps Engineer - Technical Case Study

Flight Route Discovery Assistant with MCP Tooling, Production Thinking, and Architecture
Deep Dive

Business Scenario

A major travel platform has launched an AI-powered assistant that helps users discover travel options through a conversational interface. Leadership wants to explore a similar capability and aims to launch an internal beta within 2 weeks.

Currently, the route data team has provided access to public airport, airline, and route datasets. Other teams that would be needed for richer end-to-end travel experiences, such as live schedules, pricing, booking, payments, and customer data, are not available during this prototype phase.

Leadership still wants to validate whether a conversational route-discovery experience is valuable.

The long-term vision is broader than route lookup. The system may later need to coordinate with additional services, data sources, and AI capabilities. Your design should therefore avoid assumptions that would make future expansion unnecessarily hard.

Main Focus of the Exercise

The OpenFlights dataset is intentionally simple and is provided only as a lightweight domain for the exercise.

We are not primarily evaluating complex data ingestion, data modelling, scraping, or production-grade data pipelines.

The main focus is how you design and implement an LLM-facing tool layer, preferably using MCP, that allows an assistant to safely and cleanly access route-discovery capabilities

Available Data Source

Use the OpenFlights airport, airline, and route datasets as the initial data source.

- Documentation: <https://openflights.org/data.php>
- Airports data:
<https://raw.githubusercontent.com/jpatokal/openflights/master/data/airports.dat>
- Airlines data:
<https://raw.githubusercontent.com/jpatokal/openflights/master/data/airlines.dat>

- Routes data:

<https://raw.githubusercontent.com/jpatokal/openflights/master/data/routes.dat>

Important: the route dataset is historical and should not be treated as live schedule, pricing, booking, or navigation data. This limitation is acceptable for the exercise, but the user experience and production discussion should make it clear.

Your Task

Implement a working coded prototype of a conversational flight route discovery assistant. The prototype should include:

1. An AI/LLM-facing assistant or agent layer.
2. A separate route-discovery capability exposed through an MCP server, MCP-compatible tools, or a clearly explained equivalent tool interface.
3. A clean integration boundary between the assistant and the route data source.

The assistant itself must not directly load files, query datasets, or call external systems. It should interact with route data only through the defined tool boundary.

Focus on a strong route-discovery MVP while designing for future extensibility.

Be prepared to explain:

- what you implemented
- what you intentionally deferred
- how your tool boundary works
- how the design could evolve beyond route search
- what would be needed to make this production-ready

Expected Capabilities

The assistant should be able to answer natural-language route questions such as:

“Can I fly from Berlin to Lisbon directly?”

It should support capabilities such as:

- Find airports by city, country, IATA code, or airport name.
- Find direct routes between airports or cities.
- Explain limitations of the data and avoid presenting historical route data as live availability.
- Return structured, user-friendly responses.
- Optionally suggest simple alternatives when no direct route exists.

The route-search functionality should be exposed through clearly defined tools. We expect the tool interface to be simple, typed where possible, and independently testable. Example tools could include `find_airports`, `find_direct_routes`, `suggest_alternative_routes`.

Architecture & Future Evolution

We intentionally do not prescribe how the system should evolve. In your presentation, explain how your design could be extended beyond the initial route-search capability.

- What boundaries would you introduce between the current route-search capability and future capabilities?
- Would you use tool calls, service APIs, events, queues, agent-to-agent communication, or another pattern? Why?
- How would you keep future extensions observable, secure, and independently testable?
- What would need to change if the system moved from a single assistant prototype to multiple specialized AI-enabled services?

You do not need to implement these future capabilities. We are interested in how you reason about the evolution path and the tradeoffs in your design.

Presentation Format

Please prepare a presentation and live walkthrough. The full presentation and discussion should not exceed 30 minutes.

1. **Prototype Demo (~10 min):** Demonstrate the assistant, conversational route queries, and how the implemented system retrieves route information.
2. **Architecture Walkthrough (~10 min):** Explain system boundaries, the data access/integration design, orchestration, and key decisions.
3. **Productionization & Prioritization (~10 min):** Discuss reliability, scaling, data freshness, future extensibility, evaluation, and what you would prioritize next.

Deliverables

Please submit the following at least **24 hours before the interview**:

- Runnable code or a working prototype demo.
- Short setup instructions.
- Architecture explanation or diagram.
- Brief notes on tradeoffs, limitations, and next steps.

Important Notes

- We intentionally do not prescribe the exact architecture, frameworks, storage, or orchestration tools.
- We are interested in how you reason about tradeoffs and prioritization.
- You are not expected to build a fully production-ready system in the given timeframe.
- You should think pragmatically about MVP scope versus future evolution.
- You may use AI tools. Please be ready to explain where they helped, which technical decisions you validated yourself, and which parts reflect your own judgment and prior experience.

Recommended Timebox

Please spend no more than 3 hours on the implementation and preparation. We value clarity, prioritization, and reasoning more than feature completeness. If you run out of time, prioritize a

minimal working route-discovery flow with a clean MCP or MCP-compatible tool boundary over adding more features.